RAII File Descriptor Wrapper - Resource Management

Theory, Edge Cases & Code Examples

- ✓ RAII (Resource Acquisition Is Initialization)
- ✓ File Descriptor Management
- ✓ Double Close
- ✓ Move Semantics
- ✓ Safe File Reading

→ Swipe to learn →

RAII File Descriptor Wrapper - Resource Management

Theory

📌 1. RAII (Resource Acquisition Is Initialization)

Principle: Resource lifetime bound to object lifetime.

Benefits:

- Automatic cleanup (no manual close())
- Exception-safe
- No resource leaks

Pattern:

```
class Resource {
public:
    Resource() { /* acquire */ }
    ~Resource() { /* release */ }
};

{
    Resource r;  // Acquired
    // Use resource
}  // Auto-released (destructor called)
```

RAII File Descriptor Wrapper - Resource Management

Theory

📌 2. File Descriptor Management (1/2)

C-style (manual):

```
int fd = open("file.txt", O_RDONLY);
// ... use fd ...
close(fd);  // ← Easy to forget! Leak if exception thrown
```

RAII wrapper:

```
FileDescriptor fd("file.txt", O_RDONLY);
// ... use fd ...
// Auto-closed on scope exit
```

```
#include <fcntl.h>
#include <unistd.h>
#include <stdexcept>
#include <utility>

class FileDescriptor {
private:
    int fd_;

public:
    FileDescriptor(const char* path, int flags, mode_t mode = 0644)
        : fd_(open(path, flags, mode))
    {
        if (fd_ == -1) {
            throw std::runtime_error("Failed to open file");
        }
    }

    explicit FileDescriptor(int fd = -1) : fd_(fd) {}

    ~FileDescriptor() {
        if (fd_ != -1) {
            close(fd_);
        }
    }

// ...continues on next slide
```

RAII File Descriptor Wrapper - Resource Management

Theory

📌 2. File Descriptor Management (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    FileDescriptor(const FileDescriptor&) = delete;
    FileDescriptor& operator=(const FileDescriptor&) = delete;

    FileDescriptor(FileDescriptor&& other) noexcept : fd_(other.fd_) {
        other.fd_ = -1;
    }

    FileDescriptor& operator=(FileDescriptor&& other) noexcept {
        if (this != &other) {
            if (fd_ != -1) {
                close(fd_);
            }
            fd_ = other.fd_;
            other.fd_ = -1;
        }
        return *this;
    }

    int get() const { return fd_; }

    ssize_t read(void* buf, size_t count) {
        return ::read(fd_, buf, count);
    }

    ssize_t write(const void* buf, size_t count) {
        return ::write(fd_, buf, count);
    }

    void close_fd() {
        if (fd_ != -1) {
            close(fd_);
            fd_ = -1;
        }
    }

    bool is_open() const {
        return fd_ != -1;
    }
};
```

RAII File Descriptor Wrapper - Resource Management

Edge Cases

📌 1. Double Close

```
FileDescriptor fd("file.txt", O_RDONLY);
fd.close_fd();
fd.close_fd();  // ← Safe (checks fd_ != -1)
```

RAII File Descriptor Wrapper - Resource Management

Edge Cases

📌 2. Move Semantics

```
FileDescriptor fd1("file.txt", O_RDONLY);
FileDescriptor fd2 = std::move(fd1);  // fd1 no longer owns FD

fd1.read(...);  // ← DANGER: fd1.fd_ is -1
```

RAII File Descriptor Wrapper - Resource Management

Code Examples

📌 1. Safe File Reading (1/2)

This example demonstrates exception-safe file reading using RAII to guarantee the file descriptor is always closed, even if errors occur.

What this code does:
- Opens a file using the RAII wrapper, which automatically closes it on scope exit
- Allocates a 1024-byte buffer on the stack for reading
- Reads up to 1024 bytes from the file in a single read operation
- Writes the read data to standard output if any bytes were successfully read
- The file descriptor is automatically closed when the function returns or throws an exception

Key concepts demonstrated:
- RAII ensures the file descriptor cannot leak even if an exception is thrown after opening
- No explicit close() call needed - the destructor handles cleanup automatically
- Stack-based buffer allocation is efficient for small, temporary buffers
- Early return is safe because the destructor still runs during stack unwinding

Why this matters:
- Traditional C code using open()/close() is prone to leaks if close() is forgotten or exceptions occur
- Operating systems have limited file descriptor resources (typically 1024 per process)
- File descriptor leaks accumulate over program runtime and eventually cause "too many open files" errors
- RAII makes resource management deterministic and composable with other RAII types

Real-world applications:
- Configuration file parsing where errors must not leak file handles
- Log file reading in long-running servers
- Batch file processing where thousands of files are opened/closed
- Network socket management (similar pattern applies to socket file descriptors)

```
#include <iostream>
#include <vector>

void readFile(const char* path) {
    FileDescriptor fd(path, O_RDONLY);

// ...continues on next slide
```

RAII File Descriptor Wrapper - Resource Management

Code Examples

📌 1. Safe File Reading (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    std::vector<char> buffer(1024);
    ssize_t bytes_read = fd.read(buffer.data(), buffer.size());

    if (bytes_read > 0) {
        std::cout.write(buffer.data(), bytes_read);
    }

    // Auto-closed on scope exit (even if exception)
}
```

RAII File Descriptor Wrapper - Resource Management

Code Examples

📌 2. Pipe Communication (1/2)

This example demonstrates using RAII with Unix pipes to safely manage inter-process communication file descriptors.

What this code does:

- Creates a Unix pipe (two file descriptors: read end and write end)
- Immediately closes the read end manually since this example only needs the write end
- Returns the write end wrapped in the RAII FileDescriptor object for automatic cleanup
- Writes a message to the pipe (which would be readable by another process with the read end)
- The write end is automatically closed when write_fd goes out of scope in main()

Key concepts demonstrated:

- RAII works seamlessly with operating system resources like pipes
- Pipes create two file descriptors that must both be managed correctly
- Explicit close of unused descriptors prevents resource exhaustion
- Move semantics allow transferring ownership of file descriptors between scopes safely

Why this matters:

- Pipes are fundamental for inter-process communication in Unix systems
- Each pipe consumes two file descriptors from the process's limited pool
- Forgetting to close unused pipe ends causes subtle bugs (blocked reads, descriptor leaks)
- Child processes often close pipe ends they don't need after fork()

Real-world applications:

- Shell pipelines (command1 | command2) use this mechanism internally
- Parent-child process communication after fork()/exec()
- Redirecting stdin/stdout/stderr to/from files or network sockets
- Building custom inter-process message passing systems

Performance considerations:

- Pipe writes are buffered by the kernel (typically 64KB buffer)
- Closing the write end signals EOF to readers (important for clean shutdown)
- Leaking pipe descriptors can exhaust the per-process limit (ulimit -n)

RAII File Descriptor Wrapper - Resource Management

Code Examples

📌 2. Pipe Communication (2/2)

```
FileDescriptor createPipe() {
    int pipefd[2];
    if (pipe(pipefd) == -1) {
        throw std::runtime_error("pipe() failed");
    }

    // Close read end, return write end
    close(pipefd[0]);
    return FileDescriptor(pipefd[1]);
}

int main() {
    FileDescriptor write_fd = createPipe();

    const char* msg = "Hello, pipe!";
    write_fd.write(msg, strlen(msg));

    // Auto-closed
}
```

What You Learned

Key Takeaways

- ✓ RAII (Resource Acquisition Is Initialization)
- ✓ File Descriptor Management
- ✓ Double Close
- ✓ Move Semantics

→ Swipe to learn →